basic_stats.pyCode

Site Functionality Requirements

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 pages.
- The implementation of the *State* dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing

not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in .

Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of)
- Unit tested your functions where possible, being : each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above. I have attatched my code and the requirments for the final project give a prompt that will vibe code the program I planned out to do (use the passed chats to help with making sure I hit all the things I wanted to do

…

For your CISC108 final project, I recommend building PIV Flow Analyzer, an interactive website that lets users upload Particle Image Velocimetry (PIV) data, visualize fluid flow, calculate statistics, and compare different experiments.

Based on our previous conversations, this is the project you were planning. The goal is to turn the Python functions you've already written into a working website while satisfying every requirement in your grading rubric.

Your existing `basic_stats.py` already contains useful starting code: the `PIVData` dataclass, CSV parsing, speed calculations, mean and maximum velocity calculations, and Matplotlib visualizations. It does need a few corrections before integration, particularly to the speed scatter plot, handling of empty data, and code that runs automatically when the file is imported.

I also checked the official Drafter documentation. It supports CSV uploads using `FileUpload`, embedded Matplotlib charts using `MatPlotLibPlot`, and downloading generated files through `Download`. That means you can build the core functionality within Drafter rather than switching to Flask or another framework.

Drafter Documentation

+2

Copy-and-paste prompt for your coding assistant

Attach your current `basic_stats.py` file along with the prompt below.

How this covers your rubric

The proposed structure gives you a margin above the minimum requirements:

Requirement	Planned implementation
7 routes	14 named routes
5 pages	8 functional pages
4 State fields	7 fields
4 modified State fields	6–7 fields modified
Second dataclass	`PIVData`

List of dataclass instances	`state.datasets`
3 inputs	TextBox, TextArea, SelectBox, CheckBox
3 meaningful if statements	CSV validation, dataset selection, comparison validation, filtering
Loop over dataclass instances	Displaying uploaded datasets
Drafter	All navigation, state, inputs and pages use Drafter

One thing I would prioritize

Make sure the first complete version actually works before spending time on styling. Your strongest features are the three velocity visualizations, the statistics dashboard, and comparing two PIV experiments. These are what make your project more than a generic website.

For an introductory programming final, getting those features working reliably, with proper tests and documented code, is more valuable than adding complicated functionality that could jeopardize deployment.

CISC108 Final Project – PIV Flow Analyzer

I'm working on my final project for CISC108 at the University of Delaware, and I want to turn the Python code I've already written into a complete website using Drafter. I've attached my current file, `basic_stats.py`, which contains most of the basic calculations and some visualizations I've started working on.

I want you to use my existing code as the starting point rather than completely rewriting it. You can fix mistakes and reorganize things where needed, but try to keep the code relatively simple since this is an introductory Python course.

The project is a **Particle Image Velocimetry (PIV) analysis website**. I previously worked with PIV data in fluid dynamics research, and I thought it would be interesting to build a website where people could upload their experimental data, analyze the velocity measurements, visualize the fluid flow, and compare different experiments.

The main idea is to make something that someone could use without having to write any Python themselves.

1. What I've Already Done

Please look through my attached `basic_stats.py` file before writing the website.

So far, I've written:

- A `PIVData` dataclass that stores the x and y coordinates, u and v velocity components, and the calculated speeds.
- `read_csv()` to read the PIV data and organize it into the dataclass.
- `calculate_speeds()` to calculate velocity magnitude using the horizontal and vertical velocity components.
- `mean_value()` to calculate averages.
- `max_speed()` to find the maximum velocity magnitude.
- `display_results()` to print the basic statistics.
- `display_velocity_vectors()` to plot velocity arrows using Matplotlib.
- `display_speed()` to plot velocity magnitudes.
- `display_velocity_field()` to plot velocity vectors colored by magnitude.

I want to keep these functions wherever possible, although I understand some need to be adjusted to work with Drafter.

A few things I'd like you to fix:

- Make sure the CSV reader can handle blank lines, missing values, extra whitespace, and invalid numerical data without crashing.
- Validate that the CSV has the required `x,y,u,v` columns.
- Make sure functions like `mean_value()` don't divide by zero if there isn't any data.
- Fix my `display_speed()` function so that the velocity magnitude controls the color of the points, rather than their size.
- Make the plotting functions display graphs in the Drafter website instead of opening separate Matplotlib windows.
- Remove the test code at the bottom that automatically opens a local file and displays graphs when the program runs.
- Keep the calculations in regular Python rather than switching everything over to Pandas or NumPy.

2. How I Want the Website to Work

I want the website to have a few main sections that are connected through navigation.

Home Page

This should be the starting page where someone can learn what the website does and navigate to the different features.

I'd like it to have:

- A title that says "PIV Flow Analyzer."
- A short description explaining PIV and what the website analyzes.
- Navigation to upload data, view experiments, visualize results, compare experiments, change settings, and export results.
- A simple overview of how many experiments have been uploaded.
- A message explaining how to get started if there isn't any data yet.

Upload Data Page

This is where users should be able to upload their own PIV experiment data.

The page should include:

- A file upload component that accepts a CSV file.
- A `TextBox` where the user can give the experiment a name.
- An optional `TextArea` for describing the experiment.
- A button to submit and analyze the uploaded data.

The CSV files should follow this format:

```
None
x,y,u,v

0,0,3,4

1,0,1,0

2,0,0,2
```

When the user uploads a file, the program should:

1. Read the CSV contents using my existing parsing function.
2. Check that the data is valid.
3. Calculate the velocity magnitude at each point using `sqrt(u² + v²)`.
4. Store the information in a `PIVData` instance.
5. Add that experiment to the list of uploaded datasets in State.
6. Set the new experiment as the currently selected dataset.
7. Take the user to the results page.

If the file is empty, incorrectly formatted, or contains invalid data, show an understandable error message instead of crashing.

Results and Statistics Page

Once someone uploads an experiment, I want them to see the main statistics for that dataset.

Include:

- Experiment name and description.
- Total number of velocity vectors.
- Average horizontal velocity (U).
- Average vertical velocity (V).
- Average velocity magnitude.
- Maximum velocity magnitude.
- Minimum velocity magnitude.

These values should all be calculated from the actual dataset. Reuse the functions I've written wherever possible.

I want the statistics to be clearly organized rather than printed in one block of text. There should also be links or buttons to view visualizations, compare experiments, and export data.

Visualization Page

This is probably the most important part of the project. I want to use Matplotlib to show how the fluid is moving.

I already started making these plots in my code, so build on those functions.

There should be three visualization options:

1. Velocity Vector Field

- Use `plt.quiver()`.
- Display arrows showing the direction and relative magnitude of fluid velocity.
- Use the x and y values for the positions.
- Use the u and v values for the velocity components.
- Include axis labels and a title.
- Keep the axes at equal scale.

2. Velocity Magnitude Map

- Use `plt.scatter()`.
- Plot the data points based on their x and y positions.
- Color the points according to their calculated velocity magnitude.
- Include a colorbar to show what the different colors represent.
- Make it easy to see where the fluid is moving faster or slower.

3. Color-Coded Velocity Vector Field

- Combine the first two visualizations.
- Use arrows to show direction and relative magnitude.
- Color each arrow based on the velocity magnitude.
- Include a colorbar.

For the website, I'd like:

- A `SelectBox` that allows the user to choose between these three graphs.
- The selected graph to display directly on the page using Drafter's `MatPlotLibPlot`.
- The graph to update when the user selects a different visualization or experiment.
- The graphs to respond to the visualization settings described below.

Make sure these graphs actually render inside the browser, rather than using `plt.show()`.

Experiment Management Page

I want users to be able to upload more than one experiment and switch between them.

This page should:

- Display a list of all uploaded datasets.
- Show each experiment's name, number of vectors, and average velocity magnitude.
- Let the user select an experiment to view its statistics and plots.
- Let the user delete an experiment.
- Update the selected experiment correctly if one is removed.

- Display a helpful message if there aren't any datasets.

This is also where I think we can satisfy the rubric requirement involving a loop over a list of dataclass instances.

For example, the program should use a loop that goes through `state.datasets` and accesses each experiment's attributes to display its information.

Comparison Page

One feature I really wanted from the beginning was the ability to compare different PIV experiments.

I want users to be able to:

- Choose two experiments that have already been uploaded.
- View the experiment names next to each other.
- Compare their average horizontal velocities.
- Compare their average vertical velocities.
- Compare their average velocity magnitudes.
- Compare their maximum velocities.
- See the numerical difference in average speed.
- See two velocity visualizations side by side.

For the plots, using Matplotlib subplots would be fine. I want someone to be able to look at two different flow conditions and see how the velocity fields change.

If there are fewer than two experiments, the page should explain that another experiment needs to be uploaded. It should also prevent invalid dataset selections.

Visualization Settings Page

I'd like to have a few settings that actually change the visualizations.

Minimum velocity threshold

- Use a `TextBox` where the user enters a numerical value.
- Store this value in State.
- Only display velocity vectors whose magnitudes are at least that threshold.
- Don't permanently remove the original data.
- The main statistics should still use the full dataset unless explicitly labeled as filtered statistics.
- Handle invalid and negative inputs appropriately.

Normalize velocity arrows

- Use a `CheckBox`.
- Store whether this is enabled in State.
- If selected, make the arrows represent the direction of velocity without their original magnitude.
- If not selected, use the actual u and v components.
- Handle zero-velocity vectors without dividing by zero.

I want both of these settings to be meaningful and actually affect the graphs.

Export Results Page

Finally, I want users to be able to download their results.

The export options should include:

- A CSV summary of the experiment's statistics.
- A processed CSV containing x, y, u, v, and calculated speed for every vector.
- If it's manageable, a CSV comparing the statistics from two experiments.

Use Drafter's download functionality rather than requiring users to access files directly from the Python folder.

3. Dataclasses and State

The assignment specifically requires a `State` dataclass with at least four fields, including a list or dictionary of instances of another dataclass.

I already have `PIVData`, so I'd like to continue using it. You can add a name and description to that dataclass if it makes sense.

For State, I was thinking of something similar to:

```
None
@dataclass

class State:

    datasets: list[PIVData]

    active_index: int

    compare_index: int

    plot_mode: str

    minimum_speed: float

    normalize_arrows: bool

    status_message: str
```

You can adjust this structure if needed, but please make sure it fully meets the rubric.

In particular:

- `datasets` should change when experiments are uploaded or deleted.
- `active_index` should change when a different dataset is selected.
- `compare_index` should change when someone selects a comparison experiment.
- `plot_mode` should change when the visualization is switched.

- `minimum_speed` should change when the user updates the threshold.
- `normalize_arrows` should change when the checkbox setting is updated.
- `status_message` can be used to display relevant success or error messages.

At least **four separate State fields must be meaningfully modified by routes**. Simply initializing them or assigning the same values repeatedly does not count.

Make sure State is properly initialized when the website starts. Since there won't be any uploaded data initially, use a safe way to represent that no experiment is currently selected.

Don't use global mutable variables to store uploaded data or website settings.

4. Pages and Routes

The assignment requires at least 7 routes and 5 pages. I would rather have a few extra routes so there's no question about whether I've met the requirement.

These are the routes I'm thinking of:

Route	What it should do
`index`	Display the Home page
`upload_page`	Display the CSV upload form
`process_upload`	Validate and save the uploaded data
`results_page`	Display statistics for the current dataset
`visualization_page`	Display the selected Matplotlib graph

`set_visualization`	Update which graph is selected
`datasets_page`	Show all uploaded experiments
`select_dataset`	Change the active experiment
`delete_dataset`	Remove an experiment
`comparison_page`	Display the comparison interface
`compare_datasets`	Select the experiment to compare
`settings_page`	Display visualization settings
`save_settings`	Update the user's settings
`export_page`	Display the download options

Please make sure all of these are real, working Drafter routes with the correct `@route` decorators. Navigation buttons and links need to lead to the correct pages, and form inputs need to pass their values to the appropriate routes.

It's fine if some routes handle an action and then return another page instead of displaying a completely separate page.

5. Requirements From My Professor

These are the actual site requirements for the final project. Please make sure every single one is satisfied because missing more than two requirements can result in no credit.

- At least 7 routes.

- At least 5 pages.
- A `State` dataclass with at least 4 fields.
- A second dataclass, such as `PIVData`.
- At least one State field containing a list or dictionary of instances of the second dataclass.
- At least 4 State fields, including the list or dictionary, must each be meaningfully modified in a route.
- At least 3 input fields using `TextBox`, `CheckBox`, `SelectBox`, or `TextArea`.
- At least 3 meaningful if statements.
- At least 1 loop that meaningfully iterates over a list or dictionary of dataclass instances and uses their attributes.
- The website must have a legitimate purpose and functional features.
- No global variables, except global constants.
- The website must use Drafter.
- Any advanced coding techniques not covered in CISC108 must be thoroughly documented.

I want you to specifically check that the State mutation requirement is met. It's not enough just to define the fields. Four of them actually have to change during different user interactions.

Also, don't rely on a `FileUpload` alone for the three input fields. Make sure there are at least three of the specific input components listed in the rubric.

6. Code Quality and Unit Tests

My TA will also grade the quality of the Python code, not just whether the website works.

Please follow these standards:

- Use descriptive variable and function names.
- Include type annotations in all function headers.
- Give every function a useful docstring as the first statement in the function body.

- Try to keep lines under 80 characters and never exceed 120 characters.
- Break complicated functions into smaller helper functions.
- Use ordinary Python loops and conditionals wherever possible.
- Keep the code at an introductory Python level.
- Avoid unnecessary advanced libraries and complicated syntax.
- Explain advanced techniques with comments if they are necessary.

Unit testing is especially important. The instructions say that each route should have at least one unit test where possible.

Please write meaningful tests for:

- CSV parsing, including correctly formatted and invalid files.
- Calculating velocity magnitudes.
- Calculating average and maximum velocities.
- Handling empty lists and datasets.
- Uploading and storing a dataset.
- Selecting different experiments.
- Deleting an experiment.
- Updating visualization types.
- Updating velocity thresholds.
- Enabling or disabling normalization.
- Selecting experiments for comparison.
- Calculating comparison statistics.
- Generating the exported CSV content.
- Each Drafter route that can reasonably be tested.

Use fresh State instances in tests so that one test doesn't affect another. Use Drafter's testing functions, such as `assert_state` and `assert_has`, where appropriate, or basic assertions for helper functions.

If there is something that can't reasonably be unit tested, mention that in a comment or docstring and test the underlying calculations separately.

Make sure tests have expected values and actually verify correctness rather than only checking that a function runs.

7. Design and Layout

I want the website to look like a simple scientific analysis tool rather than a basic collection of buttons.

I don't need anything extremely complicated, but I would like:

- A consistent blue, white, and gray color scheme.
- A clear title and navigation across the website.
- Well-organized statistics.
- Graphs that are large enough to read.
- Properly labeled axes and legends.
- Clear instructions for uploading data.
- Helpful messages when no experiments have been uploaded.
- Buttons that are easy to understand.
- A layout that doesn't feel crowded.

Feel free to use Drafter's styling and layout components, but don't add an entirely separate frontend framework just to make it look better.

I'd rather have a simple, functional, polished website than a visually impressive website with broken functionality.

8. Important Technical Details

Please keep these things in mind while building the project:

- Use the official Drafter documentation at https://drafter-edu.github.io/drafter/ to verify how components work.
- Make sure uploaded files are handled in the format that Drafter actually provides.

- Use `MatPlotLibPlot` correctly so graphs appear on the website.
- Don't use `plt.show()` inside the routes.
- Make sure plots don't interfere with one another when switching visualizations or comparing datasets.
- Validate indices before using them to access datasets.
- Make sure the SelectBox selections are properly converted or matched to the intended datasets.
- Make sure the program works before any experiments have been uploaded.
- Don't assume local test CSV files will always be present.
- Make sure exported files contain real calculated values.
- Don't remove or overwrite existing datasets unless the user explicitly chooses to delete one.
- Keep the raw measurements unchanged when applying visualization filters.
- Don't use Flask, Streamlit, React, or JavaScript to replace Drafter.
- Avoid adding unnecessary dependencies.

If you discover that one of my requested features is difficult to implement with Drafter, find the simplest supported solution that preserves the intended functionality. Don't silently remove a feature.

9. How I'd Like You to Build It

I think it makes sense to develop the website in the following order so it's easier to catch issues.

1. **Start with my existing code.** Review the functions, fix obvious problems, and organize the dataclasses.
2. **Get the basic website working.** Create the Home page, Upload page, and Results page. Make sure a real CSV can be uploaded and analyzed.
3. **Add visualizations.** Get all three Matplotlib plots working in Drafter.
4. **Add experiment management.** Allow multiple uploads, switching between experiments, and deleting datasets.

5. **Add comparison functionality.** Show statistical comparisons and two plots side by side.
6. **Add settings.** Implement the velocity threshold and arrow normalization controls.
7. **Add exporting.** Generate the downloadable CSV files.
8. **Finish the design and tests.** Make navigation consistent, test the routes, and clean up the code.

I want the main features to be reliable before worrying too much about visual appearance.

10. What I Want When You're Done

Please give me:

1. The complete, working Python source code for the website.
2. Any additional files that are actually needed to run it.
3. Two sample PIV CSV files with different flow patterns so I can test the comparison feature.
4. Instructions for running the project using Drafter.
5. Unit tests for the functions and routes wherever possible.
6. A final checklist explaining exactly where each grading requirement is satisfied.
7. Any important limitations or things I should know before submitting.

Don't leave placeholders or unfinished sections in the code. I want the routes connected and the features actually working.

If you can execute the program, please run it and its tests and fix any issues you find. If you can't run it, explain which parts still need to be verified rather than claiming it works.

My biggest priority is making sure this meets every requirement in the rubric while still being something I can understand and explain. I want to take the basic PIV calculations and plots I've already written and turn them into a useful website that can analyze multiple experiments, display fluid flow, and compare different conditions.

I don't want the project to become unnecessarily complicated, but I do want it to feel like a complete and useful scientific tool rather than just a class assignment.

This version keeps the structure a coding assistant can follow without the overly formal tone. It also makes the grading requirements explicit enough that the assistant can check them individually instead of overlooking one while building the website.

okay now use this prompt to code my entire projrect thank you

Your PIV Flow Analyzer project is coded, including the website, calculations, visualizations, experiment comparisons, exports, sample datasets, and unit tests. I built it using your original `basic_stats.py` as the starting point.

Complete project

Python source, tests, sample data, original code, and instructions

What's included

- Main website code — piv_flow_analyzer.py
- Unit tests — test_piv_flow_analyzer.py
- Sample straight-flow CSV
- Sample vortex-flow CSV
- Setup instructions and grading checklist

Requirements covered

Requirement	Implemented
At least 7 routes	19 routes

At least 5 pages	8 pages
State dataclass with 4+ fields	7 fields
Second dataclass	`PIVData`
List of dataclass instances	`state.datasets`
4 meaningfully modified State fields	All 7
At least 3 input fields	4 input types
3 meaningful if statements	Multiple throughout
Loop over dataclass instances	Dataset management page
Drafter library	Yes
No global mutable state	Yes
Unit tests	29 automated tests

Testing status

29 automated tests passed

The tests covered the calculations, CSV parsing, routes, dataset selection and deletion, settings, comparisons, and exports. The Matplotlib plots were also rendered successfully during testing.

One important limitation: I couldn't install the actual Drafter library in this environment, so the tests used a local compatibility shim. The code follows Drafter's official API

documentation, but you'll still need to launch it with the real library and check the browser interactions before submitting.

How to run it

After extracting the ZIP, open the project folder in your Python environment and install Drafter and Matplotlib:

```
None
python -m pip install drafter matplotlib
```

Then run:

```
None
python piv_flow_analyzer.py
```

Once it opens, try Load two sample experiments on the homepage. That will give you two datasets immediately so you can explore the graphs and comparison features.

You can also run the tests with:

```
None
python -m unittest -v test_piv_flow_analyzer.py
```

The README includes a manual testing checklist and explains exactly how each rubric requirement is fulfilled.

I recommend running the website and clicking through every page before submitting it. That final real-browser check is especially important for the upload controls, visualization settings, and download links.

piv_flow_analyzer.pyCode

test_piv_flow_analyzer.pyCode

sample_straight_flow.csvSpreadsheet

2 more

why does mean U and mean V equal 0

That's actually expected behavior for the rotating-flow sample dataset, and it doesn't mean the fluid isn't moving.

The reason is that Mean U and Mean V measure average velocity in a specific direction, not average speed.

- Mean U: Average horizontal velocity. Positive values indicate movement right, and negative values indicate movement left.
- Mean V: Average vertical velocity. Positive values indicate movement up, and negative values indicate movement down.
- Mean Speed: Average magnitude of velocity, regardless of direction.

Example: Rotating flow

Imagine four vectors in your PIV experiment:

Vector	U	V	Speed
1	2	0	2
2	-2	0	2
3	0	2	2

4	0	-2	2
Mean	0	0	2

Even though the average horizontal and vertical velocities are zero, the fluid is still moving.

This happens because positive and negative velocity components cancel each other out.

I checked the sample CSV files in your project

Here are the actual calculated results:

Statistic	Straight Flow	Rotating Flow
Mean U	1.410	0.000
Mean V	0.050	0.000
Mean Speed	1.414	0.955

The rotating-flow sample was designed to be symmetric, so its horizontal and vertical velocity components cancel out.

The average speed is still approximately 0.955, showing that the fluid is moving.

Should you change anything?

I would keep the calculations exactly as they are because they're mathematically correct.

However, I would consider adding Mean Speed as the most prominent statistic on your results page. For PIV analysis, the difference between mean velocity components and mean speed is important.

If you're seeing Mean U = 0 and Mean V = 0 for the Straight Flow sample as well, that would be unexpected, and we'd want to investigate.

If I wanted to improve this code in aspects you think its lacking what would you improve

After reviewing your actual `piv_flow_analyzer.py`, I think your project already has a solid foundation. You have CSV uploading, three visualizations, statistics, dataset management, experiment comparisons, settings, and exports.

What I think it's missing is more meaningful scientific analysis, better visualization controls, and a few improvements that make the website easier to use.

Since this is your CISC108 final project, I'd focus on improvements that make the program noticeably better without introducing code that's too advanced to explain.

My top 7 improvements

Priority	Improvement	Why it matters	Difficulty
1	Better graph controls	Makes flow visualizations easier to interpret	Easy–Medium
2	More statistics	Makes the program more useful for actual PIV analysis	Easy

3	Improved experiment comparisons	Makes comparing two flow conditions more meaningful	Medium
4	Better dashboard design	Makes the website feel more professional	Easy–Medium
5	CSV validation and upload feedback	Prevents confusing errors and crashes	Easy
6	Download visualizations	Lets users save graphs for reports	Easy–Medium
7	Better tests and reliability	Reduces the chance of problems during grading	Medium

1. Improve the visualization page

This is where I would spend the most time. Your current program has three useful graph modes, but users cannot control much about their appearance beyond the minimum-speed threshold and normalization setting.

Artificial Intelligence in Particle Image Velocimetry | Features | Feb 2020 | Photonics Spectra

Velocity vector field

03.tif (204124 bytes)

Velocity magnitude visualization

I'd improve this by adding:

- Arrow density control: Let users display every vector, every second vector, or every fourth vector. Large PIV datasets can make quiver plots extremely crowded.
- Color scale control: Allow the user to set the maximum value of the velocity colorbar.
- Show/hide grid: Add a checkbox to toggle gridlines.
- Better labeling: Allow users to specify measurement units, such as mm for position and m/s for velocity.

Arrow density is particularly useful and quite simple to implement. You could use an ordinary loop that only includes every nth data point.

2. Add more statistical analysis

Your current `display_results()` calculates:

- Mean U and Mean V
- Mean speed
- Maximum speed
- Minimum speed
- Number of vectors

Those are good starting statistics, but I would add three more.

Example of an improved statistics dashboard

Mean speed

1.414 m/s

Max speed

1.910 m/s

Speed variation

0.314 m/s

Mean U

1.410 m/s

Mean V

0.050 m/s

Vectors

9

Illustrative layout with example values, not a live calculation.

Standard deviation of speed would be my first addition. It measures how much the velocities vary around their average.

For example, two experiments might have the same mean speed, but one could have nearly uniform flow while the other has large differences across the field.

You could also add the percentage of nearly stationary vectors, using a user-defined cutoff, and the magnitude of mean velocity, calculated as:

$$|\overline{v}| = \sqrt{\overline{U}^2 + \overline{V}^2}$$

This would help explain why Mean U and Mean V can both be zero even when Mean Speed is positive, as you noticed earlier.

3. Make comparisons more scientifically useful

Your `make_comparison_plot()` already uses the same color scale for both experiments, which is good.

However, I found an issue worth improving: the graphs don't enforce identical X and Y axis limits.

That means two experiments with different spatial ranges could be displayed at different scales, making them harder to compare fairly.

I'd fix this first, then add a percentage difference in mean speed and a short summary such as:

> Experiment B has a 14.2% higher mean speed than Experiment A.

A more ambitious feature would be a third plot showing the difference in velocity magnitude at corresponding positions. This would require the two datasets to use matching measurement coordinates, so I would make it optional.

4. Improve the home dashboard

Currently, your homepage mostly shows the number of datasets and an option to load sample experiments.

I'd make it a proper dashboard with the currently selected experiment, its mean and maximum speeds, and a small preview of its velocity field.

I would also improve the workflow so the user doesn't have to navigate across multiple pages just to understand one experiment. For instance, after uploading a file, they could see their statistics and a small velocity plot immediately.

Two small usability issues I noticed in the existing code are that clicking Load two sample experiments repeatedly creates duplicate datasets, and `status_message` can continue appearing on later pages after the action is complete. Both are worth fixing.

5. Make uploading more reliable

Your current `read_csv()` has good validation, including checks for missing columns, invalid numbers, and empty datasets.

One limitation is that it manually splits each row using commas. That's fine for your sample files, but it won't correctly handle quoted CSV fields that themselves contain commas.

Using Python's built-in `csv` library would make this more robust, although you should document how it works since this is an introductory course.

I'd also display the number of successfully parsed vectors and the experiment's velocity range immediately after uploading. For large uploads, consider a reasonable file-size or vector-count limit so the website doesn't become unresponsive.

6. Let users download the actual graphs

Right now, your Export page only downloads CSV files.

For a research-focused website, being able to download a graph as a PNG would be genuinely useful. Drafter's `MatPlotLibPlot` provides a `to_picture()` method that can be used with its `Download` component.

Drafter Documentation+1

I'd add a Download graph option on both the Visualization and Comparison pages.

That would let a student generate a plot and directly include it in a lab report or presentation.

7. Do a real Drafter browser test

This one matters more for your grade than adding another feature.

The project's 29 automated tests were run using a Drafter compatibility shim rather than a real browser session. That means the route logic has been tested, but it doesn't fully establish that every interactive component works with the installed Drafter version.

I would test uploading CSV files, changing visualization modes, saving settings, deleting datasets, comparing experiments, and downloading exports in the actual website.

Also, Drafter's documentation says that state does not persist after a page reload or closing the tab. Your current app stores its experiments in State, so downloaded CSVs are important for keeping results.

Drafter Documentation

What I would prioritize for your final

If you only have time to implement three improvements, I'd choose:

1. Arrow density and graph customization

Makes the existing visualizations much more useful and demonstrates meaningful user interaction.

2. Speed standard deviation and a distribution histogram

Adds actual scientific analysis instead of just additional interface features.

3. Better side-by-side comparisons

Shared axis limits, percentage differences, and improved labels make this more valuable for experimental research.

I would not add machine learning, complicated fluid dynamics calculations, or an entirely new frontend. The strongest version of this project is a reliable PIV analysis tool with clear scientific visualizations and meaningful comparisons.

Your project is already above the rubric's minimum feature count. The next improvements should emphasize quality, usability, and scientific usefulness, rather than simply adding more pages and routes.

give a good prompt to apply these imrpovements

Here’s a prompt you can paste directly into Cursor, Claude, or another coding assistant along with your existing `piv_flow_analyzer.py` file. I kept it organized but conversational, and made sure it focuses on improving your current project rather than rebuilding it. I also included requirements to protect your CISC108 rubric and ensure the new features actually work.

Improving My CISC108 PIV Flow Analyzer Project

I've already built a working version of my PIV Flow Analyzer website for my CISC108 final project at the University of Delaware. I'm attaching my current Python code, and I want you to improve what I've already made rather than start from scratch.

The website uses the Drafter library and lets users upload Particle Image Velocimetry (PIV) CSV files, calculate velocity statistics, generate Matplotlib visualizations, manage multiple experiments, compare datasets, change visualization settings, and export results.

I think the website has a good foundation, but I want to make it more useful for scientific analysis, improve how the graphs look, and make the interface easier to navigate. I also want to fix a few things that could cause problems when someone uses the website.

Since this is an introductory Python class, please keep the code relatively simple. I need to be able to understand and explain everything I've written, especially the functions, loops, dataclasses, and how State works.

1. Improve the Visualization Controls

The visualization page is one of the most important parts of the website, and I want to give users more control over the graphs.

Please add the following features:

Arrow density control

- Add a `SelectBox` that allows users to display every vector, every second vector, every fourth vector, or every eighth vector.
- Use an ordinary loop to select which vectors are displayed rather than complicated NumPy operations.
- Store the selected density in State so the choice stays active when navigating between pages.
- Make this work for both regular vector plots and color-coded vector plots.
- Do not modify the original uploaded data.

Color scale control

- Allow users to enter an optional maximum value for the velocity colorbar.
- Store the setting in State.
- Use it when generating velocity magnitude plots and color-coded vector fields.
- If the input is empty, automatically calculate a suitable maximum using the data.
- Validate the input so negative values, zero, or nonnumerical inputs don't cause errors.
- When comparing two experiments, use the same color scale for both graphs so the results can be compared fairly.

Gridlines

- Add a `CheckBox` to show or hide gridlines on the graphs.
- Store this setting in State.
- Make sure it actually changes the displayed plots.

Measurement units

- Allow users to specify units for position and velocity, such as mm and m/s.
- These should be displayed in the axis labels, colorbars, and statistics.
- Treat these as labels only. Do not convert numerical measurements unless the user explicitly requests a conversion.
- Keep the units consistent across the website.

Please integrate these settings into the existing visualization and settings pages instead of creating unnecessary new pages.

2. Add More Useful Statistics

Right now, the results page shows the mean U, mean V, mean speed, maximum speed, minimum speed, and number of vectors.

I'd like to expand this slightly so it provides more meaningful information about the flow.

Please add:

Standard deviation of velocity magnitude

- Calculate how much the individual speeds vary from the average.
- Use the population standard deviation formula, dividing by the number of vectors.
- Write a helper function using basic Python loops and arithmetic.
- Display the result on the statistics page.

Magnitude of the mean velocity

- Calculate this using the mean horizontal and vertical velocities:

$$
|\overline{V}|=\sqrt{\overline{U}^2+\overline{V}^2}
$$

- Display this separately from the mean speed.
- Make sure the labels clearly distinguish the two values.

This is important because, for a rotating flow, Mean U and Mean V can both equal zero even though the fluid is still moving.

Percentage of nearly stationary vectors

- Use a user-defined velocity cutoff to determine which vectors are considered nearly stationary.
- Calculate the percentage of vectors with speeds below that cutoff.
- Make the cutoff adjustable and clearly explain what it represents.
- Keep this separate from the visualization's minimum-speed filter so the two settings don't get confused.

Velocity distribution histogram

- Add a new visualization option that displays a histogram of velocity magnitudes.
- Use Matplotlib.
- Label the axes clearly and display the selected velocity units.
- Show how many vectors fall into each speed range.
- Include the histogram in the existing visualization selection rather than creating another page.

Please organize these new statistics so the results page still looks clean and isn't overcrowded.

3. Improve Experiment Comparisons

One of the main purposes of the website is comparing two experiments, so I want this feature to be more useful.

Please make the following improvements:

- Make sure both plots use identical X and Y axis limits when they are shown side by side. This is especially important if the experiments have different spatial ranges.
- Keep the same color scale for both experiments when using color-coded plots.
- Add the percentage difference in mean velocity magnitude.
- Display the difference in maximum speed.
- Compare the standard deviation of speed.
- Make it obvious which experiment is A and which is B.

- Add a short summary explaining the main numerical differences between the two experiments.

For example, the website could display:

"Experiment B has a 14.2% higher mean speed than Experiment A."

The summary should be generated from the actual data, not hardcoded.

If the original mean speed is zero, handle that situation without dividing by zero. In that case, show the absolute difference and explain that a percentage change cannot be calculated.

If it's not too complicated, I'd also like an optional third comparison plot that shows the difference in velocity magnitude at corresponding positions.

Only generate that plot if the datasets have matching coordinates. If their coordinate grids don't match, explain why the difference plot isn't available instead of producing incorrect results.

Please use straightforward loops and basic Python logic for this feature.

4. Improve the Home Dashboard

The homepage currently has some basic information about the uploaded datasets, but I'd like it to feel more like an actual analysis dashboard.

Please update it to include:

- The name of the currently selected experiment.
- The number of uploaded experiments.
- Mean speed and maximum speed for the selected experiment.
- A small preview of the selected experiment's velocity field.
- Clear buttons for uploading data, viewing results, comparing experiments, and accessing visualizations.

If no experiments have been uploaded, display a simple introduction and instructions for getting started.

Also, please fix these usability issues:

- Clicking the button to load sample experiments multiple times should not keep adding duplicate copies of the same datasets.
- Success and error messages shouldn't remain visible on unrelated pages indefinitely.
- Make it easier for users to understand which experiment is currently selected.
- Make navigation consistent across all pages.

I don't want a completely different design, just a cleaner and more useful version of the existing website.

5. Improve CSV Uploading and Validation

The website already handles basic CSV uploads, but I'd like it to be more reliable.

Please improve the CSV reader by:

- Using Python's built-in `csv` library if appropriate.
- Supporting extra whitespace and blank lines.
- Validating that x, y, u, and v are present.
- Checking that each row has valid numerical values.
- Rejecting NaN and infinite values.
- Handling files with missing or incorrectly formatted data.
- Preventing empty files from being treated as valid datasets.
- Providing clear error messages that explain what went wrong and, when possible, identify the affected row.
- Avoiding uploads that are too large for the website to handle comfortably.

After a successful upload, show:

- The experiment name.
- Number of vectors imported.
- Minimum and maximum speed.
- A confirmation that the dataset was added.

If an upload fails, don't change or delete any of the experiments already stored in State.

Also, make sure the website doesn't require any particular CSV file to exist on the computer before it can start.

6. Add the Ability to Download Graphs

Right now, the website mainly exports numerical CSV files.

I would also like users to be able to save the actual graphs they generate.

Please add:

- A way to download the selected visualization as a PNG image.
- A way to download the two-experiment comparison plot.
- Proper graph titles, axis labels, and colorbars in exported images.
- A reasonable image resolution so graphs are usable in reports or presentations.
- Useful filenames based on the experiment names and visualization type.

Use functionality supported by Drafter and Matplotlib. Avoid introducing unnecessary libraries or complicated file handling.

The exported graph should reflect the user's selected settings, including arrow density, velocity threshold, color scale, gridlines, and normalization.

7. Improve the Interface and Design

I'd like to make the website look a little more professional without making the code too complicated.

Keep the general scientific dashboard style, with a blue, white, and gray color scheme.

Some things I'd like improved:

- Make the navigation consistent across all pages.

- Use clear section headings.
- Display statistics in organized groups or cards.
- Make sure graphs are large enough to read.
- Add short explanations next to settings so users understand what they do.
- Clearly indicate which dataset is currently active.
- Use consistent spacing, labels, and button names.
- Display helpful messages when an action cannot be completed.
- Avoid overcrowding pages with too many controls.

Use Drafter's existing components and styling options wherever possible.

I don't want Flask, Streamlit, React, or JavaScript added to the project.

8. Keep the Code Appropriate for CISC108

This is really important. Even though I want the website to be improved, I don't want the code to become unnecessarily advanced.

Please:

- Keep the existing `PIVData` and `State` dataclasses.
- Preserve my original functions where practical.
- Use normal loops, lists, conditionals, and helper functions.
- Include type annotations in function headers.
- Add useful docstrings to every function.
- Choose clear variable names.
- Keep lines below 120 characters.
- Break down overly complicated functions.
- Document any advanced techniques that weren't covered in class.
- Avoid unnecessary external libraries.
- Do not use global mutable variables.

You can add State fields for the new settings, but every field should have a meaningful purpose.

Also, make sure changes to State are handled through proper Drafter routes rather than through global variables.

9. Make Sure Every Original Rubric Requirement Still Passes

My professor's requirements haven't changed, and I don't want these improvements to accidentally break anything that already meets the rubric.

Please verify that the updated program still has:

- At least 7 working Drafter routes.
- At least 5 functional pages.
- A `State` dataclass with at least 4 fields.
- A second dataclass, `PIVData`.
- A list of `PIVData` objects inside State.
- At least 4 State fields, including the dataset list, meaningfully modified by routes.
- At least 3 inputs using `TextBox`, `TextArea`, `SelectBox`, or `CheckBox`.
- At least 3 meaningful if statements.
- At least one loop over `state.datasets` that accesses attributes of the `PIVData` objects.
- No global mutable variables.
- Drafter used for the entire website.
- Documentation for advanced coding techniques.
- Functions with appropriate type annotations and docstrings.
- Unit tests for each route where possible.
- A website that launches and works without syntax errors or missing dependencies.

Please explicitly check that at least four State fields are modified through actual user interactions. I don't want the rubric satisfied only superficially.

10. Testing and Reliability

I want you to expand the existing unit tests to cover the new functionality.

Include tests for:

- Standard deviation calculations.
- Magnitude of mean velocity.
- Nearly stationary vector percentages.
- Arrow density selection.
- Velocity filtering.
- Arrow normalization, including zero-speed vectors.
- Valid and invalid color scales.
- CSV validation and invalid numerical values.
- Dataset comparisons.
- Percentage changes when the original mean speed is zero.
- Matching and mismatching coordinate grids.
- Dataset deletion and active-index updates.
- Preventing duplicate sample datasets.
- Generating export files.
- Updated State values after changing settings.
- Each new or modified Drafter route wherever unit testing is possible.

Please keep the existing tests that already work.

If possible, run all tests and launch the website to catch actual errors.

I understand that automated tests don't necessarily prove that everything works in a browser, so please distinguish between what has been tested and what still needs manual verification.

11. How I'd Like You to Approach the Changes

Please make the improvements to my existing code in a manageable order.

1. Review the current source code and identify anything that needs to change.
2. Fix existing bugs or reliability issues first.
3. Add the new statistics and histogram.
4. Add the visualization settings and controls.
5. Improve the experiment comparison page.
6. Update the home dashboard and navigation.
7. Improve CSV validation.
8. Add PNG graph downloads.
9. Improve the overall layout.
10. Update the unit tests and verify the final rubric requirements.

Try to keep the code organized so it's easy to follow.

If you come across a feature that's significantly more complicated than expected, prioritize getting the existing features working correctly rather than breaking them to add something optional.

12. What I Want When You're Finished

Please provide:

- The complete updated Python program.
- Updated unit tests.
- Any additional sample CSV files needed to demonstrate new features.
- Updated instructions for running the website.
- A summary of the improvements you made.
- A checklist showing how the final program satisfies the grading rubric.
- An explanation of anything that couldn't be implemented or fully tested.

Don't leave incomplete functions or placeholders.

Most importantly, **improve the existing program instead of rebuilding it from scratch**. I want to retain the work I've already done and expand its functionality in ways that make the PIV analysis more useful.

The end result should be a clean, reliable scientific analysis website that I can comfortably explain to my TA and that demonstrates what I've learned in CISC108.

I'd rather have a smaller number of well-implemented improvements than a bunch of complicated features that don't work properly.

I've implemented the improvements in a new version of your PIV Flow Analyzer, while keeping the original project files intact.

The upgraded program now includes:

- Four visualization types: velocity vectors, speed map, colored vectors, and a new speed histogram.
- More scientific statistics: speed standard deviation, magnitude of mean velocity, and percentage of nearly stationary vectors.
- Better graph controls: arrow density, color scale, gridlines, normalization, and measurement-unit labels.
- Improved comparisons: shared plot scales, percentage differences, and a spatial speed-difference map when coordinates match.
- Better usability: clearer upload errors, a dashboard preview, no duplicate sample experiments, and downloadable PNG graphs.

43 automated tests passed, including actual Matplotlib PNG rendering.

These tests used a local Drafter compatibility layer. They verify the calculations and route logic, but a final test in the real Drafter browser is still necessary.

Download your updated project

PIV Flow Analyzer — Improved Version

Complete Python project, tests, four sample CSVs, documentation, and graph preview

Download complete project ZIP

You can also access the files separately:

- Updated Python application
- Unit tests
- Setup instructions and rubric checklist
- Preview of the comparison graph

How to run it

Extract the ZIP, open the project folder in your Python environment, and run:

```
None
```

```
python -m pip install -r requirements.txt
```

```
python piv_flow_analyzer.py
```

To run the tests:

```
None
```

```
python -m unittest -v test_piv_flow_analyzer.py
```

Final rubric check

19

Drafter routes

8

Functional pages

13

State fields

The source also meets the required dataclass, input, conditional, loop, and code-organization criteria. The README identifies where each requirement is satisfied.

Before submitting: Launch the website with the real Drafter package, upload the sample CSVs, and check the graph downloads and forms. That final browser check remains the only major verification step I couldn't complete here.

piv_flow_analyzer.pyCode

test_piv_flow_analyzer.pyCode

README.mdDocument

comparison_preview.pngImage

it says that the file piv_flow_analyzer.py is not able to upload

I checked the file, and `piv_flow_analyzer.py` is present and intact. The problem may be that the platform you're uploading it to doesn't accept `.py` files.

I've made three versions so you can try a different format:

- Python file — piv_flow_analyzer.py
- ZIP file containing just the Python code
- TXT version of the complete code

If you're uploading it to BlockPy: Try the TXT version. You can open it, copy all the code, and paste it into your Python editor. The TXT file contains the same code as the `.py` version.

If you're running the PIV website: You don't upload `piv_flow_analyzer.py` into the website. That file is the program itself. You run it in Python, then upload `.csv` experiment files through its interface.

One question: Where are you trying to upload the file—BlockPy, VS Code, or somewhere else? That will help me identify the exact problem.

piv_flow_analyzer.pyCode

piv_flow_analyzer_source.txtDocument

I checked the file, and `piv_flow_analyzer.py` is present and intact. The problem may be that the platform you're uploading it to doesn't accept `.py` files.

I've made three versions so you can try a different format:

- Python file — piv_flow_analyzer.py
- ZIP file containing just the Python code
- TXT version of the complete code

If you're uploading it to BlockPy: Try the TXT version. You can open it, copy all the code, and paste it into your Python editor. The TXT file contains the same code as the `.py` version.

If you're running the PIV website: You don't upload `piv_flow_analyzer.py` into the website. That file is the program itself. You run it in Python, then upload `.csv` experiment files through its interface.

One question: Where are you trying to upload the file—BlockPy, VS Code, or somewhere else? That will help me identify the exact problem.

piv_flow_analyzer.pyCode

piv_flow_analyzer_source.txtDocument

could you maybe need to rename since I already uploaded a file named the same

Yes! That could definitely be the issue. Some platforms don't allow you to upload two files with the same name.

I've renamed your updated project file without changing any of the code.

Download piv_flow_analyzer_updated_v2.py

Try uploading this version instead.

If it still doesn't work, let me know which platform you're uploading it to and what error message appears.

piv_flow_analyzer_updated_v2.pyCode

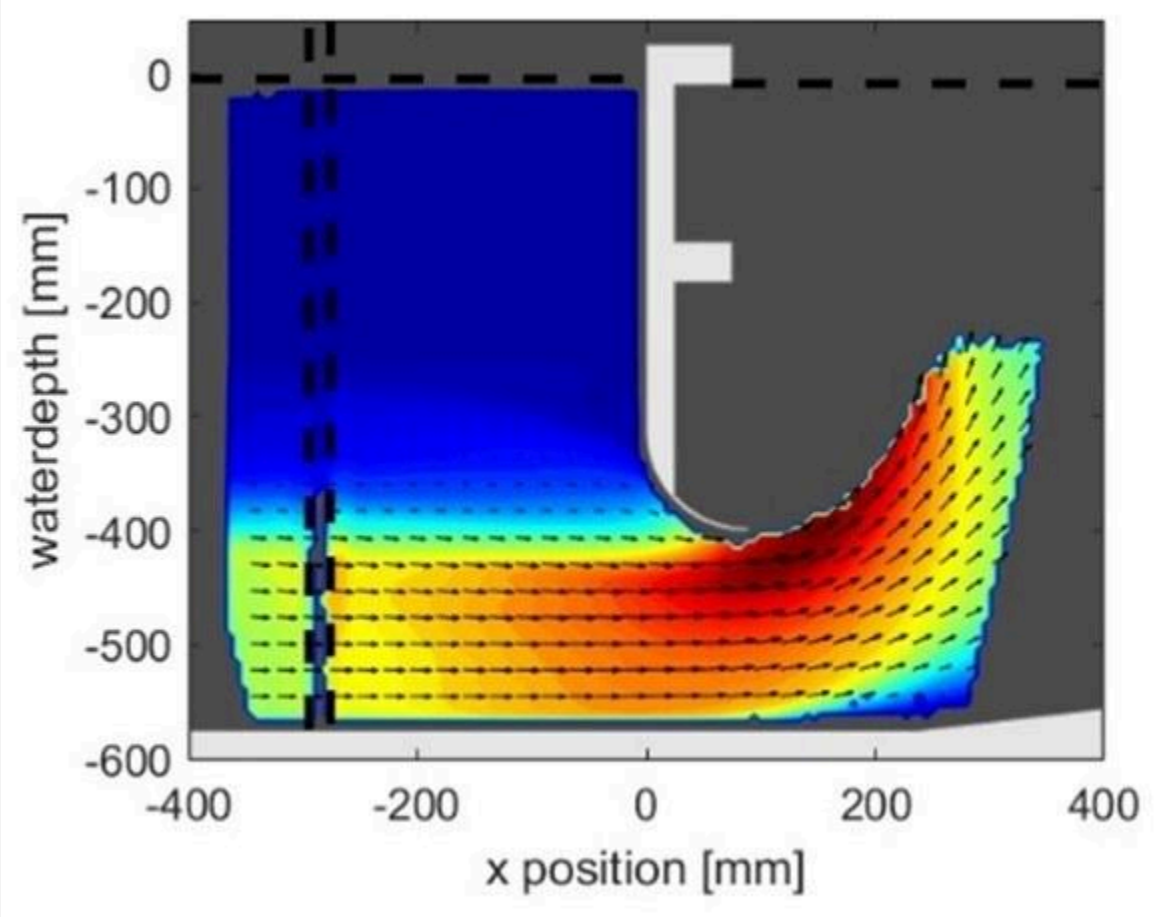
is there a way to make a heat map like the ones shown in the Improve visualization page part of your response to improve the program

Yes! You can absolutely add a smooth velocity heatmap to your PIV Flow Analyzer, similar to the scientific fluid-flow visualizations I showed earlier.

Your current speed map uses `plt.scatter()`, which colors individual measurement points. A heatmap would go further by showing a continuous color distribution across the flow field.

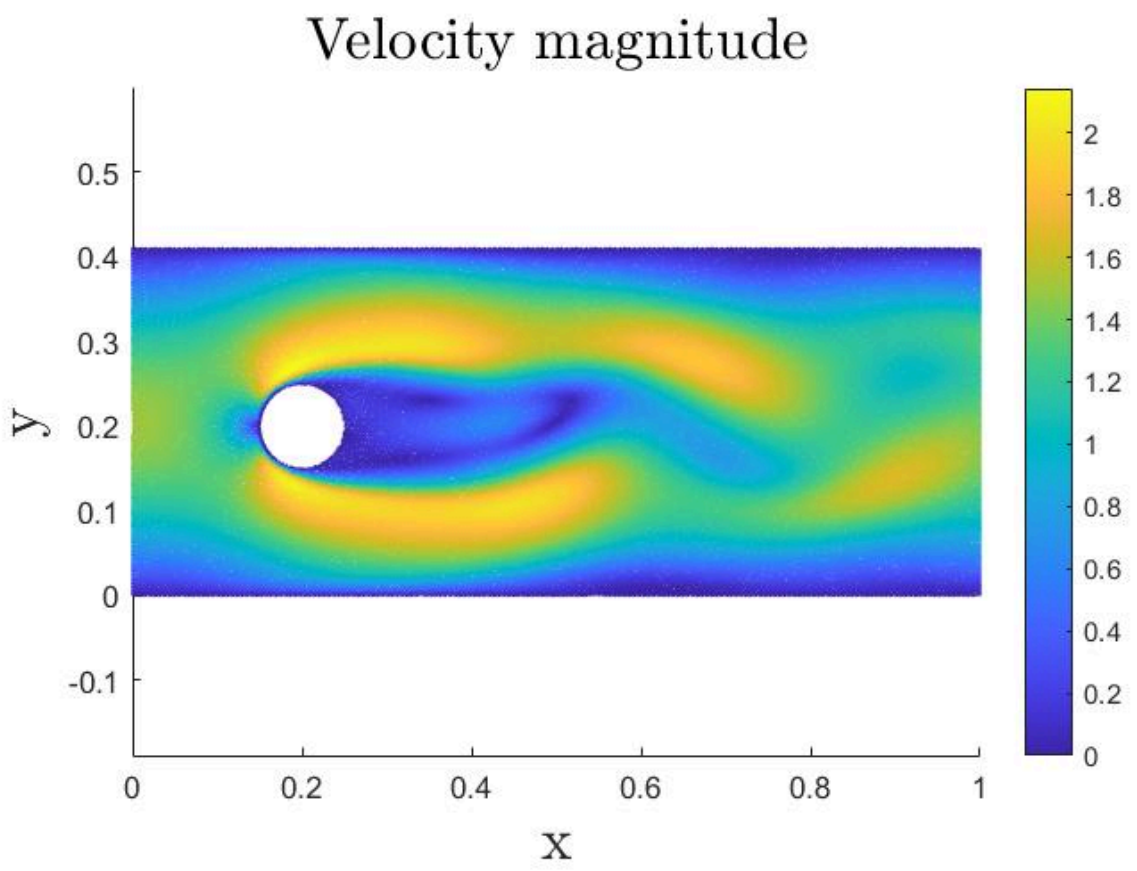
For your project, I'd recommend adding it as a fifth visualization option rather than replacing any of your existing graphs.

What the heatmap would look like



Current speed map

Individual measurement points colored by velocity magnitude.



Proposed heatmap

A smooth color field representing velocity magnitude across the measured region.

The colors would represent the fluid's speed, with cooler colors indicating slower flow and warmer colors indicating faster flow.

How I would implement it

Because your CSV already contains x, y, u, and v, you have the data needed.

The simplest option for your current project is Matplotlib's `tricontourf()` function. It can generate filled contours from scattered PIV measurement points without requiring you to reshape the data into a rectangular grid.

```
None
def display_velocity_heatmap(data: PIVData) -> None:    """Display a filled heatmap of velocity magnitude."""    plt.figure(figsize=(8, 6))    heatmap = plt.tricontourf(        data.x,        data.y,        data.speeds,        levels=20,        cmap="turbo"    )    plt.colorbar(heatmap, label="Velocity Magnitude")    plt.xlabel("X Position")    plt.ylabel("Y Position")    plt.title("PIV Velocity Magnitude Heatmap")    plt.axis("equal")
```

This works well when you have enough non-collinear measurement points to form triangles.

For a typical PIV grid, it can produce a visually smooth plot. It also works with irregularly spaced measurement locations.

Adding it to your Drafter website

You would need to make three changes:

1. Add `"Velocity Heatmap"` to the visualization `SelectBox`.
2. Update your plot-selection function so that choosing this option calls `display_velocity_heatmap()`.
3. Render the resulting Matplotlib figure using Drafter's `MatPlotLibPlot`, just as you do with your existing graphs.

You should also apply your existing velocity threshold, color-scale settings, and measurement-unit labels to the heatmap.

One important detail: `tricontourf()` estimates values between measured points. The smooth colors are interpolated, not additional velocity measurements. It also needs at least three distinct, non-collinear positions, and certain datasets (such as points all along one line) cannot produce a contour heatmap. In those cases, your website should display an explanation and fall back to the existing scatter plot.

An even better option for your project

I would consider adding two related visualizations:

- Velocity Magnitude Heatmap: A smooth colored background showing where the fluid moves faster and slower.
- Velocity Heatmap with Flow Arrows: The same heatmap, but with velocity vectors overlaid so users can see both speed and direction.

How to Solve a Classic CFD Benchmark: The Lid-Driven Cavity Problem | COMSOL Blog

comsol.com

The second option would be particularly useful for your project because a heatmap alone cannot show flow direction.

My recommendation: Add both as visualization options, with the arrows controlled by your existing density and normalization settings. That would make the PIV Flow Analyzer much closer to a practical fluid-dynamics analysis tool while keeping the implementation manageable for CISC108.

there is PLOT_MOdes though how do I add it in the visualizatoin page

Yes! In your current code, `PLOT_MODES` already controls the options displayed on the Visualization page. You don't need to create another dropdown or route. You just need to add the heatmap as an option and tell your plotting function how to draw it.

I've also made these changes directly to your project.

Download updated Python file with heatmap — v3

What I changed

1. Add the heatmap to `PLOT_MODES`

Near the top of your file, change it to:

```
None
PLOT_MODES = (    "Velocity vectors",    "Speed map",    "Colored vectors",    "Speed histogram",    "Velocity heatmap")
```

Your `visualization_page()` already contains:

```
None
SelectBox("plot_choice", list(PLOT_MODES), state.plot_mode)
```

That means Velocity heatmap automatically appears in the dropdown.

2. Add the heatmap to `draw_velocity_plot()`

Find the function `draw_velocity_plot()`.

Inside its conditional statements, add this branch after the `"Speed map"` case, before the final `else` that draws arrows:

```
None
elif state.plot_mode == "Velocity heatmap":    try:        art = axis.tricontourf(            visible.x,            visible.y,
```

```
visible.speeds,            levels=20,            cmap="turbo",
vmin=0,            vmax=limit        )    except (ValueError,
RuntimeError):        art = axis.scatter(            visible.x,
visible.y,            c=visible.speeds,            cmap="turbo",
vmin=0,            vmax=limit        )    axis.figure.colorbar(
art,        ax=axis,        label="Speed (" + state.velocity_units
+ ")"    )
```

This uses `tricontourf()` to create a smooth heatmap. If the data cannot form a contour, it falls back to a scatter plot. The downloadable version also handles datasets with too few points or identical speeds.

One other necessary change: replace the existing vector-sampling condition with:

None

```
if state.plot_mode in ("Velocity vectors", "Colored vectors"):
visible = sample_vectors(visible, state.arrow_density)
```

That way, your heatmap uses all available points instead of dropping measurements based on the arrow-density setting.

Here's the result

mscm.gitlab.io

The actual generated heatmap from your updated code is available in the preview file below.

View your actual heatmap preview

Do you need to change `visualization_page()`?

No. Your existing `set_visualization()` route already updates `state.plot_mode`, and `make_single_plot()` uses that selection when generating a graph.

The process is now:

User selects "Velocity heatmap"

`set_visualization()` updates `state.plot_mode`

`draw_velocity_plot()` calls `tricontourf()`

Heatmap displays on the Visualization page

I ran the existing 43 automated tests, and they passed with the updated code. I also generated heatmaps with regular, triangular, collinear, and constant-speed datasets. These checks used the local Drafter test shim, so you should still verify the dropdown in your actual browser.

This adds a fifth visualization without changing your existing Drafter pages or sacrificing any rubric requirements.

piv_flow_analyzer_heatmap_v3.pyCodepiv_heatmap_grid_preview.pngImage

does this updated file have all the code

Yes! I verified that `piv_flow_analyzer_heatmap_v3.py` contains the complete updated program, not just the new heatmap code.

It includes all your previous functionality:

- CSV uploading and validation
- PIV statistics and calculations
- All five visualization modes, including the new heatmap
- Experiment management and deletion
- Side-by-side experiment comparisons
- Visualization settings
- CSV and PNG exports
- Home dashboard
- Drafter routes and State dataclasses

The original improved version had 1,018 lines, and the new heatmap version has 1,039 lines. The new code was added to the existing program.

Download the complete updated program — v3

You can use this file to replace your previous `piv_flow_analyzer.py`.

Important: If you're running the file directly, the new filename is fine. If another file imports `piv_flow_analyzer` by name, or your unit tests expect that filename, rename the updated file to `piv_flow_analyzer.py` after replacing the old one.

Your existing sample CSV files and unit tests are separate files, so keep those as well.

piv_flow_analyzer_heatmap_v3.pyCode

give a good CSV file with lots of points that will test my code well

I've created a 4,800-point PIV CSV dataset specifically to test your Flow Analyzer, especially the new velocity heatmap.

Download your test data

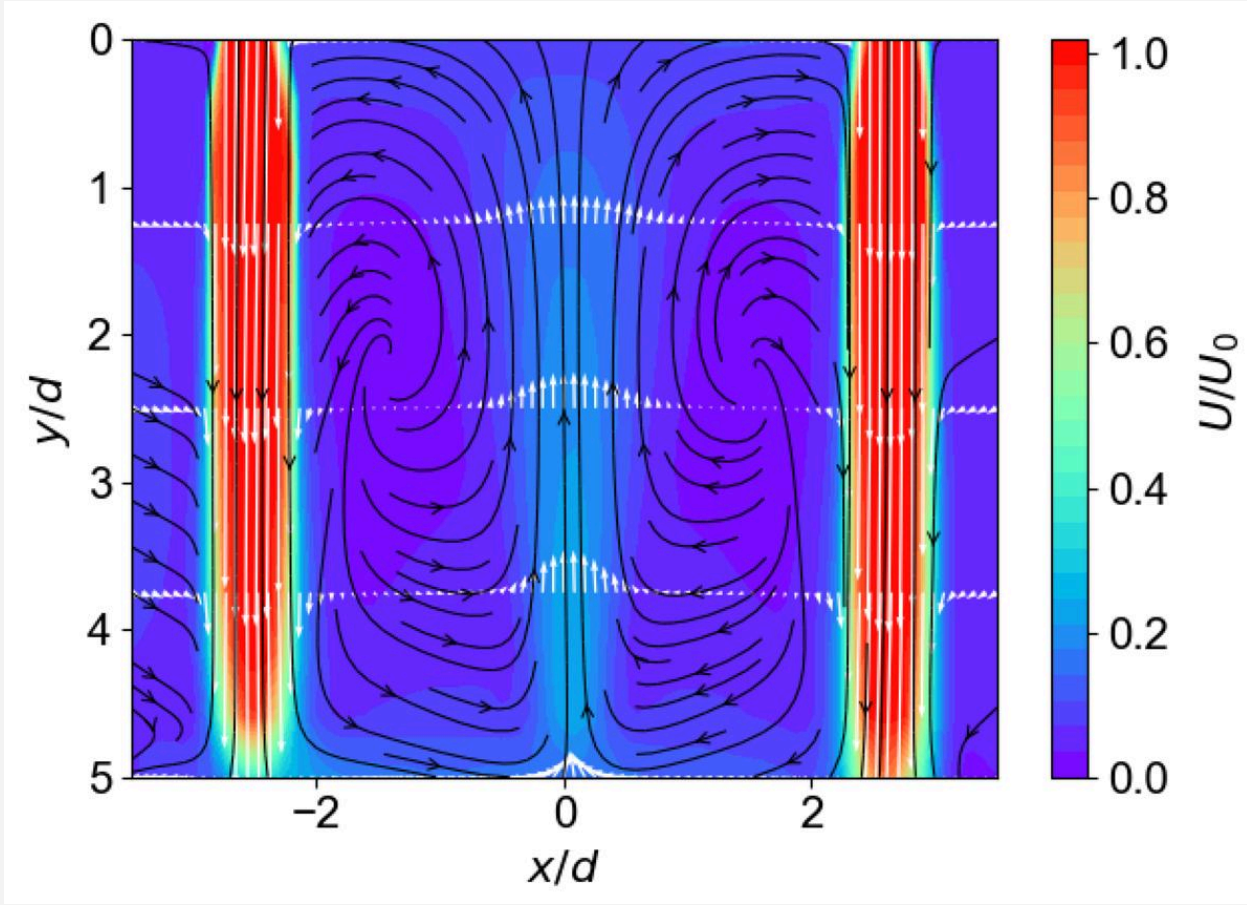
Download PIV Test CSV — 4,800 Points

Main test dataset · 125 KB

Download second CSV for comparisons

Download both datasets as a ZIP

What your heatmap should look like



View the actual heatmap generated by your program

This dataset simulates a flowing fluid with multiple vortices, a faster central stream, and slower surrounding areas.

Expected results

I ran your program's CSV reader and plotting functions against the generated data.

Measurement	Expected value
Number of vectors	4,800
Mean U velocity	0.749 m/s
Mean V velocity	~0.000 m/s
Mean speed	0.798 m/s
Maximum speed	2.065 m/s
Minimum speed	0.177 m/s
Speed standard deviation	0.394 m/s

How to test your website

1. Open your Drafter website and navigate to Upload CSV.
2. Upload `PIV_Test_Flow_4800_Points.csv`.
3. Check the statistics against the table above.
4. Navigate to Visualizations and select Velocity heatmap.
5. Test the other plot modes. For velocity arrows, try an arrow density of `8` to avoid overcrowding.

You can also upload the second CSV and use your Compare Experiments page. It contains the same 4,800 spatial coordinates but slightly different velocities, so you can test your comparison graphs and speed-difference calculations.

Note: These are synthetic fluid-flow measurements, not experimental PIV data. I verified that they load and generate plots successfully with your latest code using the local testing setup; the browser upload itself still needs to be tested.

PIV_Test_Flow_4800_Points_Heatmap_Preview.pngImage

PIV_Test_Flow_4800_Points.csvSpreadsheet

PIV_Test_Flow_4800_Points_Comparison.csvSpreadsheet

this was the heat map for the Download PIV Test CSV — 4,800 Points file is this good or bad

This is a very good result! Your PIV Flow Analyzer appears to be generating the heatmap correctly from the 4,800-point test dataset.

I'd rate it 9/10 for visualization quality, especially for a CISC108 final project.

What's working well

Smooth velocity contours

The colors transition smoothly between regions, which indicates that the contour interpolation is working.

Clear flow features

The faster regions around (50, 45) and (105, 80), along with the slower region around (135, 40), are clearly visible.

Correct labels and colorbar

The position axes are labeled in millimeters, while the colorbar shows speed in m/s. This is appropriate if those are the units of your CSV data.

High-resolution dataset

The heatmap looks detailed and does not show obvious holes or large interpolation artifacts.

Two improvements I'd recommend

1. Fit the graph more closely to your dataset.

Your actual data appears to cover approximately X = 0–160 mm and Y = 0–118 mm, but your graph has substantial empty space around the edges.

You could improve the presentation by setting the plotting limits based on the data:

```
axis.set_xlim(min(data.x), max(data.x))

axis.set_ylim(min(data.y), max(data.y))
```

Add a small margin if desired. This would make better use of the plotting area.

2. Add optional flow arrows over the heatmap.

Your heatmap shows *how fast* the fluid moves, but not *which direction* it moves.

Overlaying a small number of velocity vectors would let you see both:

```
axis.quiver(
   data.x[::20],
   data.y[::20],
   data.u[::20],
   data.v[::20],
   color="black"
)
```

For 4,800 points, sampling every 20th vector gives approximately 240 arrows, although you may want fewer for a cleaner image.

One scientific detail

The red regions represent high speed, not necessarily turbulence. Similarly, the blue regions indicate lower speeds, not necessarily laminar flow. You would need additional analysis to characterize turbulence or flow regime.

Overall assessment

Feature	Assessment
CSV data processing	Appears successful
Heatmap generation	Excellent
Color scaling	Good
Axis labels	Good
Plot boundaries	Could improve
Scientific usefulness	Very good

My recommendation: Keep this heatmap as one of your five visualization options. It is already a strong feature for your final project.

The next improvement I would prioritize is adding an option to overlay velocity arrows on the heatmap. That would make your application more useful for actual PIV analysis, rather than just making the plot look nicer.